

Summer Training Program WEEK 1 - DAY 3

- Hashing
- More about multivalued containers
- Reference copy operation (shallow copy)
- More on lists
- Sets
- Dictionaries

Hashing

Hashing is a concept that uses the hash function to determine the memory location of a variable.

Hash function : $\text{hash}(x)$

Memory address: $x \bmod N$

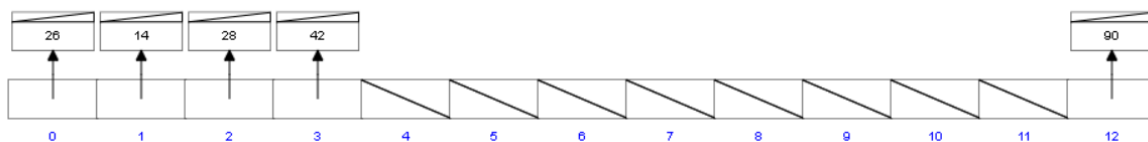
Where x: value of variable, and N : number of buckets

Eg: say we want to place '26' in the memory and we have a list of size 13

$$26 \bmod 13 = 0$$

Hence, 26 is placed at the 0th memory location.

☒ Hash Integer
☐ Hash Strings



Multivalued containers- List

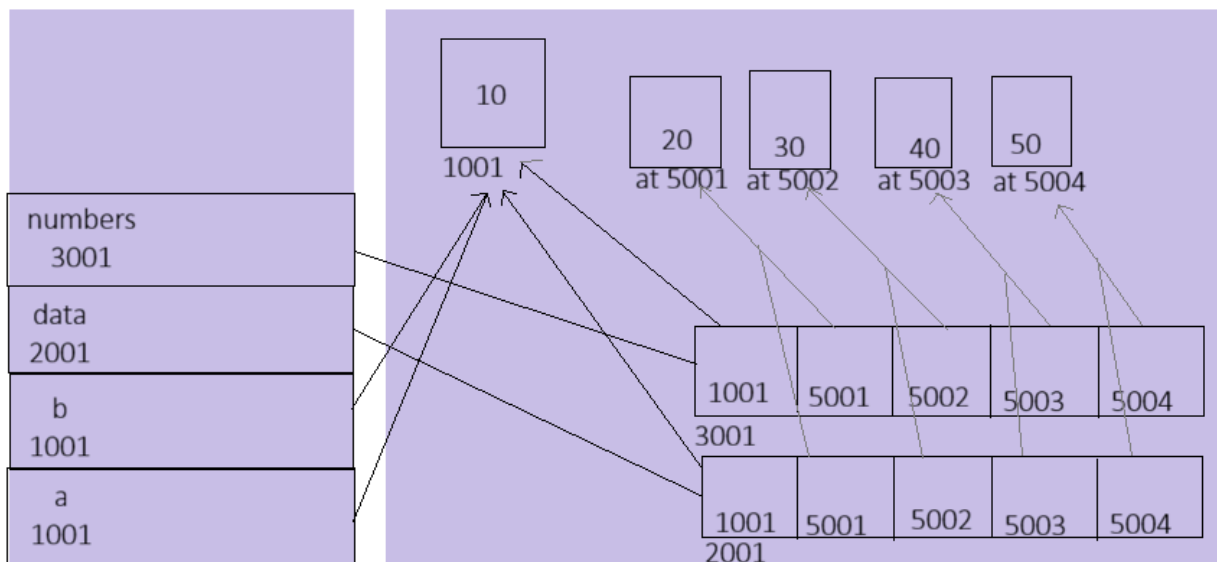
CODE:

```
1 a = 10
2 b = 10
3 data = [10,20,30,40,50]
4 numbers = [10,20,30,40,50]
5
6
7 print('a:', a, "Hash of a:", id(a), "Type of a:", type(a))
8 print('b:', b, "Hash of b:", id(b), "Type of b:", type(b))
9 print('data:', data, "Hash of data:", id(data), "Type of data:", type(data))
10 print('numbers:', numbers, "Hash of numbers:", id(numbers), "Type of numbers:", type(numbers))
```

OUTPUT:

```
a: 10 Hash of a: 140720292234440 Type of a: <class 'int'>
b: 10 Hash of b: 140720292234440 Type of b: <class 'int'>
data: [10, 20, 30, 40, 50] Hash of data: 2594341544064 Type of data: <class 'list'>
numbers: [10, 20, 30, 40, 50] Hash of numbers: 2594341692736 Type of numbers: <class 'list'>
```

MEMORY REPRESENTATION:



Important Facts:

1. `a` and `b` both have the same value 10 (single valued) hence they both have the same hashcode.
2. Even though `data` and `numbers` hold the same list, they point to different containers (this is because multivalued containers are not shared.)
3. Now if we alter `data`, `numbers` will not be altered. This is a bug in software solutions , for eg: if we open two windows of whatsapp, they both should reflect the same changes. A solution to this bug is shallow copy (reference copy operation).

Reference copy operation:

We use the assignment operator and refer one variable to another.

CODE:

OUTPUT:

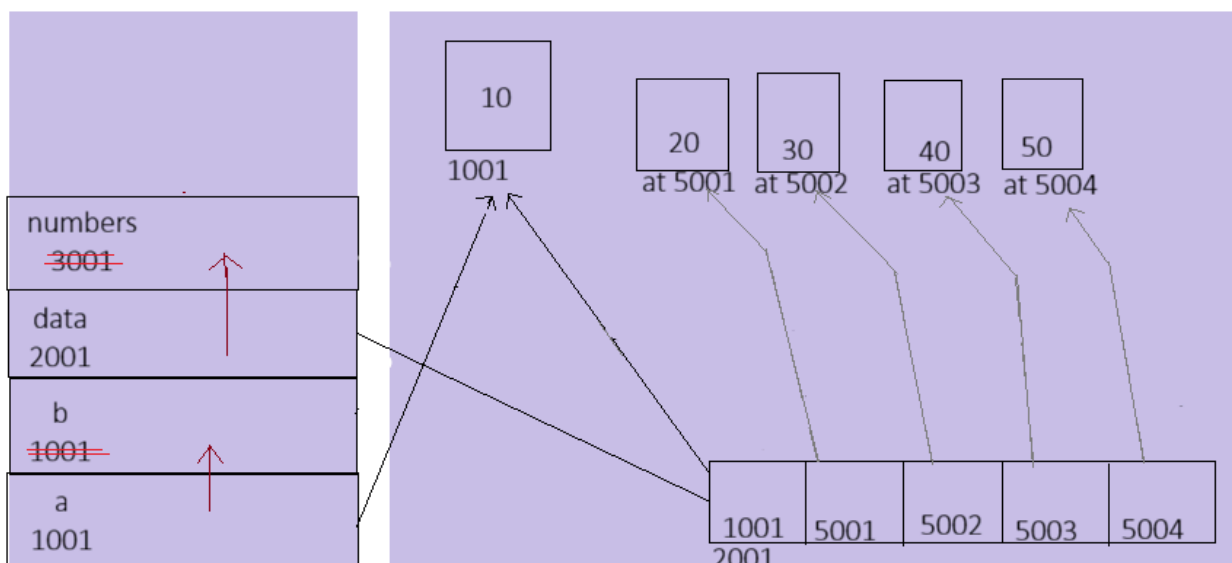
```
3 a=10
4 b=a
5
6 print("ID of a: ", id(a))
7 print("ID of b: ", id(b))
8
9 b=100
10 print("b: ", b, id(b))
11 print("a: ",a, id(a))
12
13 data=[10,20,30,40,50]
14 numbers=data
15
16 print("id of data: ", id(data))
17 print("id of numbers: ", id(numbers))
18
19 data[1]=1000
20 print('data[1]: ', data[1])
21 print('numbers[1]: ', numbers[1])
22
23 numbers[3]=4000
24 print('numbers[3]: ',numbers[3] )
25 print('data[3]: ',data[3] )
```

```
ID of a: 140720770188488
ID of b: 140720770188488
b: 100 ID of b: 140720770191368
a: 10 ID of a: 140720770188488
id of data: 1831391420544
id of numbers: 1831391420544
data[1]: 1000
numbers[1]: 1000
numbers[3]: 4000
data[3]: 4000
```

NOTE: Reference copy variable does not have a data type.

It acts the same as a void pointer like in c++.

MEMORY REPRESENTAION:



Important facts

1. In the above example, the value of a is reference copied into the value of b, so a and b now point to the same value. This works similar to the basic working of single valued containers.
2. But with the multivalued containers , data and number now refer to the same container. Here data is referenced into variables.
Any change made to data will reflect into numbers and vice versa
This is the perfect use case for reference copy operation.

TASK: Identify where reference copy operation is used in real life:

In applications like ZOMATO, the user view, restaurant view and the driver's view must all be the same- here shallow copy is used.

Other data structures:

1. Set:

Eg: setA={10,20,30,40}

Sets can be made in python using set() - function.

{ } braces do not make a set but a dictionary by default in python.

Sets are not indexed, rather they use indexing

CODE:

```
3 data = {10, 20, 30, 40, 50}
4 print('data:', data)
5 print("Hash of data:", id(data), "Type of data:", type(data))
6
7
8 numbers = {10, 20, 30, 40, 50, 10, 20}
9 print('numbers:', numbers)
10 print("Hash of numbers:", id(numbers), "Type of numbers:", type(numbers))
```

OUTPUT:

```
data: {50, 20, 40, 10, 30}
Hash of data: 2022006129376 Type of data: <class 'set'>
numbers: {50, 20, 40, 10, 30}
Hash of numbers: 2022006413504 Type of numbers: <class 'set'>
```

Important facts:

01. Sets donot support duplicate data. Even when duplicate data is input it is not stored.
02. Sets are hashed so they are not stored in the input order.
03. But ordered sets do exist, which retain the input order, they are not default though.

2. Dictionaries:

These store data in the form of a key-value pair.

Keys are unique and they point the respective value(can be duplicate).

CODE:

```
3  students = {
4      101: 'adam',
5      102: 'eve',
6      103: 'cain',
7      104: 'abel',
8  }
9
10 print('students:', students,)
11 print("Hash of students:", id(students), "Type of students:", type(students))
```

OUTPUT:

```
students: {101: 'adam', 102: 'eve', 103: 'cain', 104: 'abel'}
Hash of students: 2440316575808 Type of students: <class 'dict'>
```

NOTE:

Instead of storing the hashcode of the single valued containers, the multivalued containers store the keys.

Accessing values:

CODE:

```
1  menu = {
2      101: {
3          'name': 'Dal Makhani',
4          'price': 100
5      },
6
7      201: {
8          'name': 'Paneer Butter Masala',
9          'price': 300
10     },
11
12     301: {
13         'name': 'Burger',
14         'price': 120
15     },
16
17     401: {
18         'name': 'Noodles',
19         'price': 200
20     },
21
22     501: {
23         'name': 'Mojto',
24         'price': 80
25     }
26 }
27
28 print('MENU')
29 print(menu[101]['name'], '\u20b9', menu[101]['price'])
30 print(menu[201]['name'], '\u20b9', menu[201]['price'])
31 print(menu[301]['name'], '\u20b9', menu[301]['price'])
32 print(menu[401]['name'], ' \u20b9', menu[401]['price'])
33 print(menu[501]['name'], ' \u20b9', menu[501]['price'])
```

OUTPUT:

```
MENU
Dal Makhani ₹ 100
Paneer Butter Masala ₹ 300
Burger ₹ 120
Noodles ₹ 200
Mojto ₹ 80
```

Unicode:

- Every character other than the one's on our keyboard have a unicode so we can use them in programming.
- For eg: the unicode for rupees is 20b9 we can use it as '\u20b9', this will print the rupee symbol. Similarly many other characters can be used.
- Every file has a unicode range and every language has a unicode range, hence we can judge the language of a file by the unicode range.

INTERVIEW QUESTIONS:

- Deep copy and shallow copy
- Do reference variables have a type?

ASSIGNMENT:

1. Write the code for the following makemytrip.com as a list of dictionaries

Flights from New Delhi to Zurich, and back

**MMTSECUREINT**
Get upto 15% OFF on your flight bookin...

**ICICIINTEMI**
Use code ICICIINTEMI and get upto 12...

Popular
₹ 1,56,593 | 08h 45m

Cheapest
₹ 50,995 | 23h 45m

Onward

**Air India**
AI 7713

01:05
DEL

08h 45m
Non stop

06:20
ZRH

Return

**Swiss**
LX 2646

12:35
ZRH

08h
Non stop

00:05
DEL^{+1 DAY}

 +2 Amenities

 More flight details

2. Write the code for the following bookmyshow.com as a dictionary of list of dictionaries

F

E

D

C

B

A

₹380 PRIME

14 13 12 11 10 09 08 07 06 05 04 03  

11 10 09 08 07 06 05 04 03 02 01

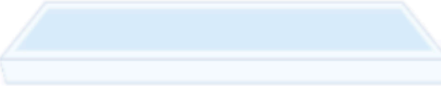
13 12 11 10 09 08 07 06 05 04 03 02 01

₹300 CLASSIC

13 12 11 10 09 08 07 06 05 04 03 02 01

13 12 11 10 09 08 07 06 05 04 03 02 01

13 12 11 10 09 08 07 06 05 04 03 02 01



All eyes this way please

CODE 1.

```
1  Flights = [  
2      {  
3          'airline': 'Air India',  
4          'airplane': 'AI 7713',  
5  
6          'From': 'New Delhi',  
7          'To': 'zurich',  
8  
9          'Departure': '2026-06-30 1:05',  
10         'Arrival': '2026-06-30 6:20',  
11         'Duration': '8h 45m',  
12  
13         'Price': 50000,  
14         'stops': 0  
15     },  
16     {  
17         'airline': 'Swiss',  
18         'airplane': 'LX 2646',  
19  
20         'From': 'Zurich',  
21         'To': 'zNew Delhi',  
22  
23         'Departure': '2026-07-15 12:35',  
24         'Arrival': '2026-07-16 00:05',  
25         'Duration': '8h',  
26  
27         'Price': 50000,  
28         'stops': 0  
29     },  
30  
31 ],  
32  
33 ]
```

CODE 2:

```
1  tickets = {
2      'A' : [
3          {
4              'seat_no': 1,
5              'avaibility': 'avaliable',
6              'cateogry': 'classic',
7              'type': 'normal',
8              'price': 300
9          },
10         {
11             'seat_no': 2,
12             'avaibility': 'avaliable',
13             'cateogry': 'classic',
14             'type': 'normal',
15             'price': 300
16         }
17     ], #upto seat 13
18 },
19
20
21  'B' : [
22      {
23          'seat_no': 1,
24          'avaibility': 'avaliable',
25          'cateogry': 'classic',
26          'type': 'normal',
27          'price': 300
28      },
29      {
30          'seat_no': 2,
31          'avaibility': 'avaliable',
32          'cateogry': 'classic',
33          'type': 'normal',
34          'price': 300
35      }
36  ], #upto seat 13
37 }
```

```
39  #similary row C
40
41  'D' : [
42      {
43          'seat_no': 1,
44          'avaibility': 'avaliable',
45          'cateogry': 'prime',
46          'type': 'normal',
47          'price': 380
48      },
49      {
50          'seat_no': 2,
51          'avaibility': 'avaliable',
52          'cateogry': 'prime',
53          'type': 'normal',
54          'price': 380
55      }
56  ], #upto row 13
57
58  ],
59
60  'E' : [
61      {
62          'seat_no': 1,
63          'avaibility': 'avaliable',
64          'cateogry': 'prime',
65          'type': 'normal',
66          'price': 380
67      },
68      {
69          'seat_no': 2,
70          'avaibility': 'avaliable',
71          'cateogry': 'prime',
72          'type': 'normal',
73          'price': 380
74      }
75  ], #upto row 11
```

```
78  'F' : [
79      {
80          'seat_no': 1,
81          'avaibility': 'avaliable',
82          'cateogry': 'prime',
83          'type': 'Special_needs',
84          'price': 380
85      },
86      {
87          'seat_no': 2,
88          'avaibility': 'avaliable',
89          'cateogry': 'prime',
90          'type': 'special needs',
91          'price': 380
92      }
93  ], #upto row 14
94
95  ]
96 }
```

GURASSIS KAUR
g.assiskaur@gmail.com